

Chapter 8

Sprint 8 – Teaching Software to Work in the Background

Understanding Asynchronous Apex in Salesforce

Objective

The objective of this sprint was to understand how Salesforce performs background processing using Asynchronous Apex. The chapter introduced different asynchronous processing mechanisms, their working principles, advantages, and appropriate use cases. It also explained how these features improve application performance, user experience, and scalability while working within Salesforce Governor Limits.

Introduction

Salesforce applications often perform operations that require significant processing time, such as sending emails, updating thousands of records, integrating with external systems, or generating reports. Executing these operations immediately can increase response time for users. Asynchronous Apex allows such operations to execute in the background, improving system performance and providing a better user experience. This chapter explains the different asynchronous processing options available in Salesforce and the scenarios where each one should be used.

Synchronous Processing

Synchronous processing executes tasks one after another in a single transaction. The user waits until the entire operation is completed before receiving a response. This approach is suitable for simple operations that require an immediate result, such as saving a record or validating user input. However, long-running operations may increase response time and reduce application performance.

Asynchronous Processing

Asynchronous processing allows operations to execute in the background after the current transaction finishes. Users do not have to wait for these operations to complete, resulting in faster response times and improved application performance. This approach is commonly used for sending emails, processing large datasets, integrating with external systems, and executing scheduled business processes.

Comparison Between Synchronous and Asynchronous Processing

Feature	Synchronous Processing	Asynchronous Processing
Execution	Executes immediately	Executes in the background
User Wait Time	User waits until completion	User continues working while processing happens later
Performance	Suitable for small tasks	Suitable for long-running tasks
Resource Usage	Higher during execution	Better resource utilization
Typical Examples	Record Save, Validation	Email Notifications, Batch Processing, Integrations

Future Methods

Future Methods are one of the simplest asynchronous Apex features. They execute background operations after the current transaction is completed. Future Methods are commonly used for callouts, notifications, and lightweight background tasks. They improve application responsiveness by postponing operations that do not require immediate execution.

Queueable Apex

Queueable Apex provides greater flexibility than Future Methods. It supports complex business logic, allows developers to pass objects between jobs, and enables job chaining where one asynchronous job starts another. Queueable Apex is recommended for most new asynchronous implementations because it offers improved monitoring and control.

Batch Apex

Batch Apex is designed to process very large numbers of records efficiently. Instead of processing all records at once, Salesforce divides the records into smaller batches. Each batch executes independently, reducing Governor Limit issues and allowing applications to process thousands or even millions of records reliably. Batch Apex consists of the **Start**, **Execute**, and **Finish** methods.

Scheduled Apex

Scheduled Apex enables developers to execute Apex classes automatically at specific dates and times. It is useful for recurring operations such as daily maintenance, weekly reporting, monthly cleanup, and periodic business processes. Scheduling ensures important background tasks run automatically without user intervention.

Choosing the Appropriate Asynchronous Mechanism

Different asynchronous tools are designed for different requirements.

Requirement	Recommended Feature
Simple background processing	Future Method
Structured asynchronous job	Queueable Apex
Processing thousands of records	Batch Apex
Time-based execution	Scheduled Apex

This selection helps developers build scalable applications while efficiently utilizing Salesforce resources.

Best Practices

- Use asynchronous processing only for operations that do not require an immediate response.
 - Choose Queueable Apex for new asynchronous implementations whenever appropriate.
 - Use Batch Apex for processing large datasets.
 - Schedule recurring operations using Scheduled Apex.
 - Keep asynchronous jobs efficient to avoid unnecessary resource consumption.
 - Design applications considering Governor Limits and scalability.
-

Key Learning Outcomes

- Understood the difference between synchronous and asynchronous processing.
- Learned how background processing improves Salesforce application performance.

- Studied Future Methods, Queueable Apex, Batch Apex, and Scheduled Apex.
 - Learned when each asynchronous mechanism should be used.
 - Understood how asynchronous processing supports scalable enterprise applications.
 - Gained knowledge of Salesforce best practices for background processing.
-

Interview Questions

1. What is Asynchronous Apex?

Asynchronous Apex allows operations to execute in the background after the current transaction completes, improving application performance and user experience.

2. What is the difference between Future Method and Queueable Apex?

Future Methods are suitable for simple background operations, whereas Queueable Apex supports complex processing, job monitoring, and job chaining.

3. When should Batch Apex be used?

Batch Apex should be used when processing a large number of records that cannot be handled efficiently in a single transaction.

4. What is Scheduled Apex?

Scheduled Apex allows Apex classes to execute automatically at specified dates and times.

5. Why is asynchronous processing important?

It reduces user waiting time, improves application performance, supports scalability, and efficiently utilizes Salesforce resources.

Conclusion

This sprint introduced the fundamentals of Asynchronous Apex and explained how Salesforce executes background operations efficiently. Understanding Future Methods, Queueable Apex, Batch Apex, and Scheduled Apex provides a strong foundation for building scalable enterprise applications. These concepts help developers design high-performance Salesforce solutions that deliver a better user experience while complying with Governor Limits and platform best practices.

Engineering Sprint 19 – Moving Secondary Work to Queueable Apex

Task – Implementing Queueable Apex for Appointment Processing

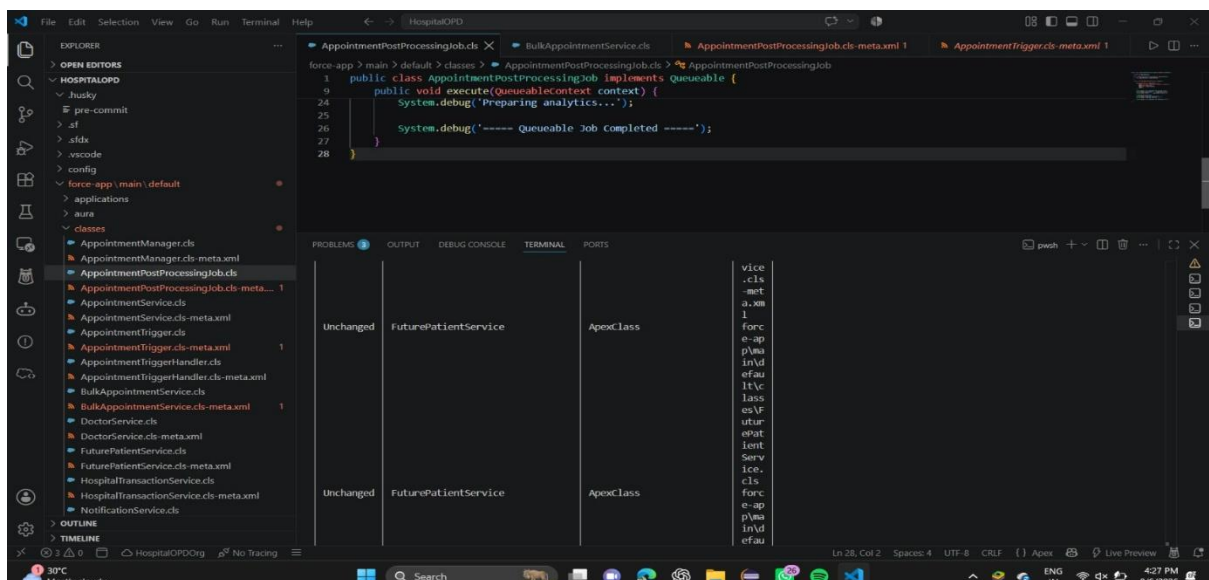
Objective

To implement a Queueable Apex class that performs secondary processing after an Appointment record is updated.

Work Performed

A Queueable Apex class named **AppointmentPostProcessingJob** was created by implementing the **Queueable** interface. The class contains the **execute()** method, where background processing activities are performed. Debug statements were added to simulate operations such as notification processing, analytics preparation, and successful job completion. This implementation demonstrates how long-running tasks can be separated from the main transaction while maintaining clean and reusable application architecture.

VS Code – Queueable Apex Class



Task – Executing the Queueable Job

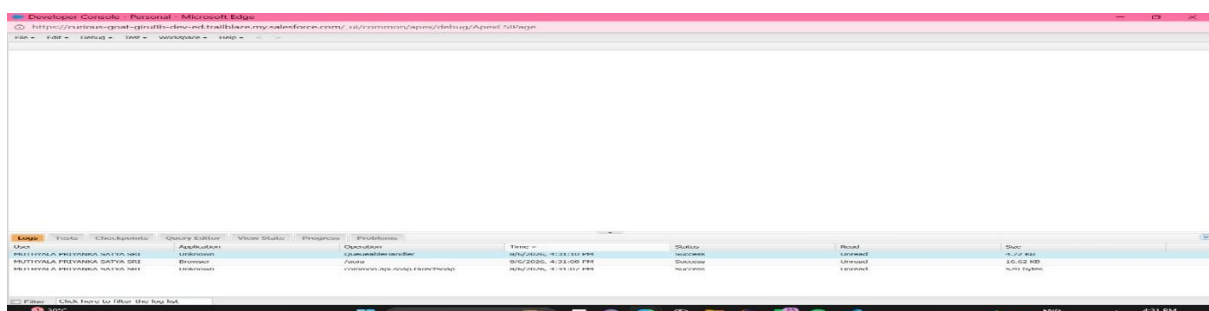
Objective

To verify that the Queueable Apex job executes successfully after being enqueued.

Work Performed

The Queueable job was executed from the Salesforce application. The execution details were monitored using the Developer Console, confirming that the asynchronous job was successfully queued and processed without errors.

Developer Console – Queueable Job Execution



Task – Verifying Background Processing

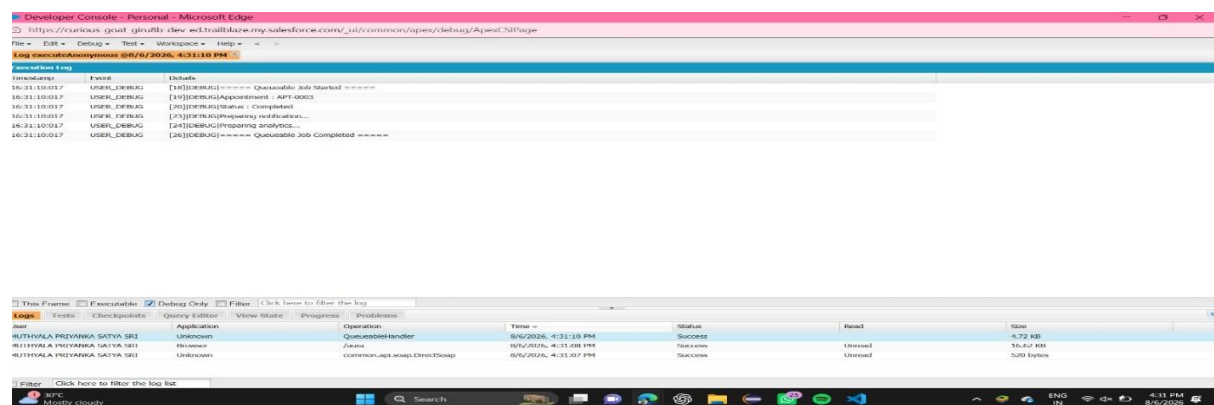
Objective

To confirm that the background operations defined in the Queueable Apex class executed successfully.

Work Performed

The execution logs displayed debug messages indicating the complete flow of background processing. The log confirmed that the Appointment information was received successfully, notification processing started, analytics preparation executed, and the Queueable job completed without any failures. This verified the correct implementation of asynchronous processing using Queueable Apex.

Developer Console – Debug Log Output



Engineering Sprint 20 – Designing a Queueable Chain

Task – Implementing Queueable Chain

Objective

To implement a Queueable Apex process that performs background synchronization of Appointment records with an external hospital system.

Work Performed

A Queueable Apex job was created to execute the synchronization process asynchronously. The Queueable class received the Appointment details, initiated the synchronization process, and simulated communication with an external hospital system using debug statements. This implementation demonstrated how Queueable Apex can be used to perform independent background operations without affecting the user transaction.

Developer Console – Queueable Job Execution

User	Application	Operation	Time	Status	Read	Size
MUTHIALA PRIVANKA SATYA SRI	Unknown	QueueableHandler	8/6/2026, 4:31:10 PM	Success	Unread	4.72 KB
MUTHIALA PRIVANKA SATYA SRI	Unknown	common.apex.job.DirectSoap	8/6/2026, 4:31:07 PM	Success	Unread	520 bytes

Task – Verifying Queueable Chain Execution

Objective

To verify the successful execution of the Queueable Apex job using Salesforce Debug Logs.

Work Performed

The Queueable job was executed successfully, and the Developer Console logs confirmed the complete execution flow. The debug messages indicated that the external hospital synchronization process started, the Appointment record was synchronized successfully, and the asynchronous job completed without any errors. This verified the successful implementation of Queueable Apex for background processing.

Timestamp	Event	Details
16:54:25:032	USER_DEBUG	[18] DEBUG ===== External Hospital Sync Started =====
16:54:25:032	USER_DEBUG	[19] DEBUG Synchronizing Appointment : APT-0006
16:54:25:032	USER_DEBUG	[22] DEBUG Appointment data synchronized successfully.
16:54:25:071	USER_DEBUG	[27] DEBUG ===== External Hospital Sync Completed =====

Engineering Sprint 21 – Processing Historical Applications with Batch Apex

Task – Creating the Batch Apex Class

Objective

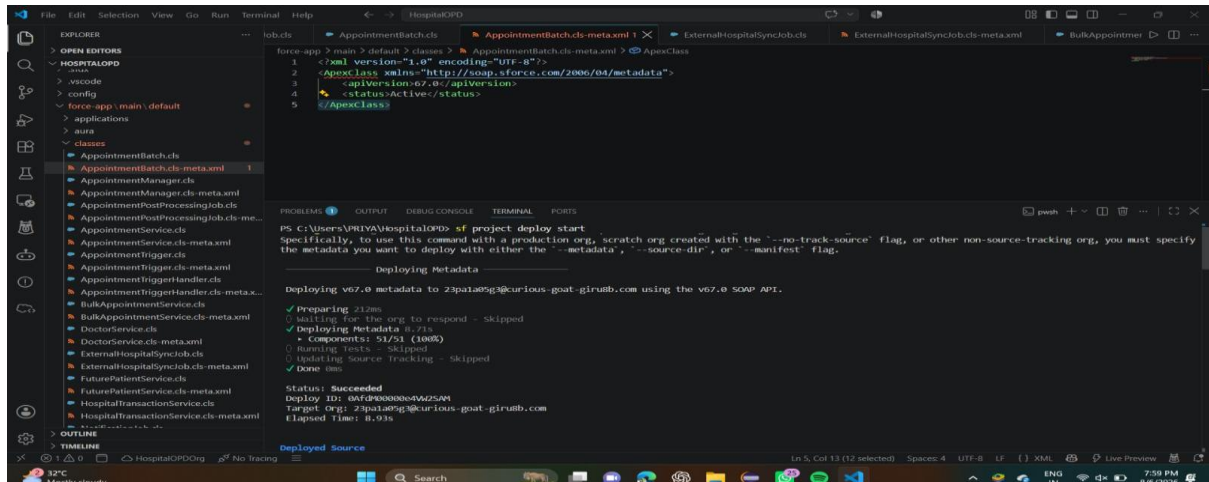
To implement a Batch Apex class for processing Appointment records asynchronously.

Work Performed

A Batch Apex class named **AppointmentBatch** was created and deployed successfully to the Salesforce organization. The batch class was designed to process Appointment records in

smaller groups instead of a single transaction. This implementation demonstrated how Salesforce handles large-scale record processing while maintaining application performance and resource efficiency.

VS Code – Batch Apex Deployment Success



Task – Executing the Batch Apex Job

Objective

To execute the Batch Apex job and verify successful batch processing.

Work Performed

The Batch Apex class was executed using Salesforce Developer Console. The execution logs confirmed that the Batch Apex job and Serial Batch Range Chunk Handler processed the records successfully. This verified that the batch process executed asynchronously and completed without any errors.

Developer Console – Batch Apex Execution

User	Application	Operation	Time	Status	Read	Size
MUTHIALA PREYANKA SATYA SRJ	Unknown	Batch Apex	8/6/2026, 8:02:10 PM	Success	Unread	3.73 KB
MUTHIALA PREYANKA SATYA SRJ	Unknown	SerialBatchApexRangeChunkHandler	8/6/2026, 8:02:10 PM	Success	Unread	3.23 KB
MUTHIALA PREYANKA SATYA SRJ	Unknown	SerialBatchApexRangeChunkHandler	8/6/2026, 8:02:09 PM	Success	Unread	5.08 KB
MUTHIALA PREYANKA SATYA SRJ	Unknown	SerialBatchApexRangeChunkHandler	8/6/2026, 8:02:09 PM	Success	Unread	2.47 KB
MUTHIALA PREYANKA SATYA SRJ	Unknown	Batch Apex	8/6/2026, 8:02:08 PM	Success	Unread	3.87 KB
MUTHIALA PREYANKA SATYA SRJ	Unknown	/services/data/v67.0/tooling/executeIn...	8/6/2026, 8:02:06 PM	Success	Unread	3.04 KB

Task – Verifying Batch Completion

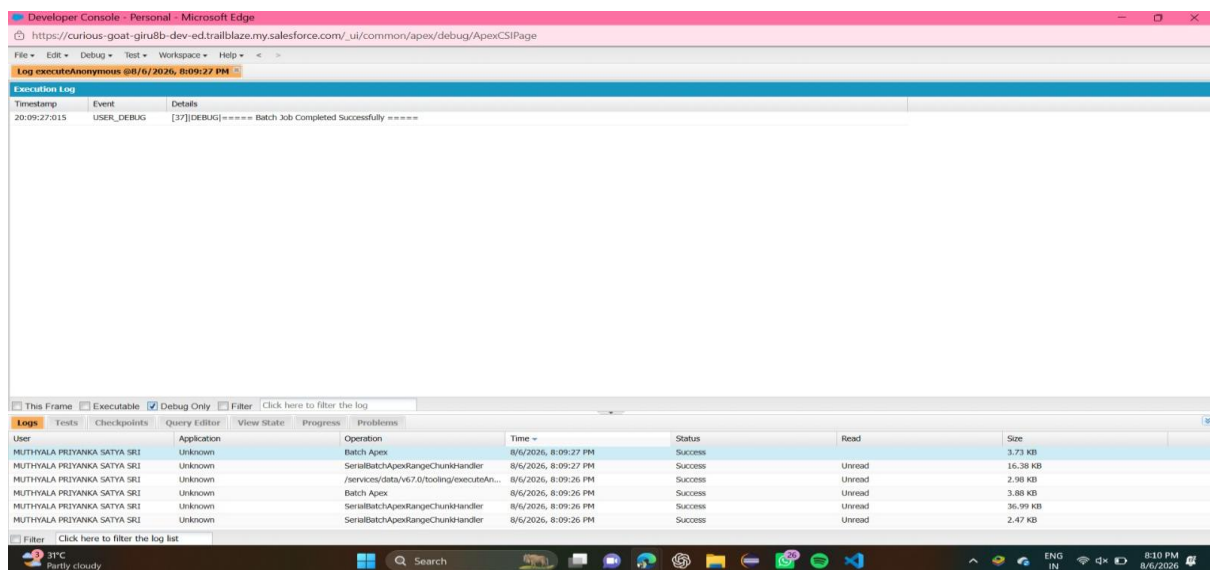
Objective

To confirm successful completion of the Batch Apex process.

Work Performed

The Developer Console debug logs displayed confirmation messages indicating that the Batch Apex job completed successfully. The successful execution verified that the batch processed the required records and completed the asynchronous transaction without failures.

Developer Console – "Batch Job Completed Successfully"



Engineering Sprint 22 – Scheduling Expired Job Processing

Task – Scheduling the Apex Job

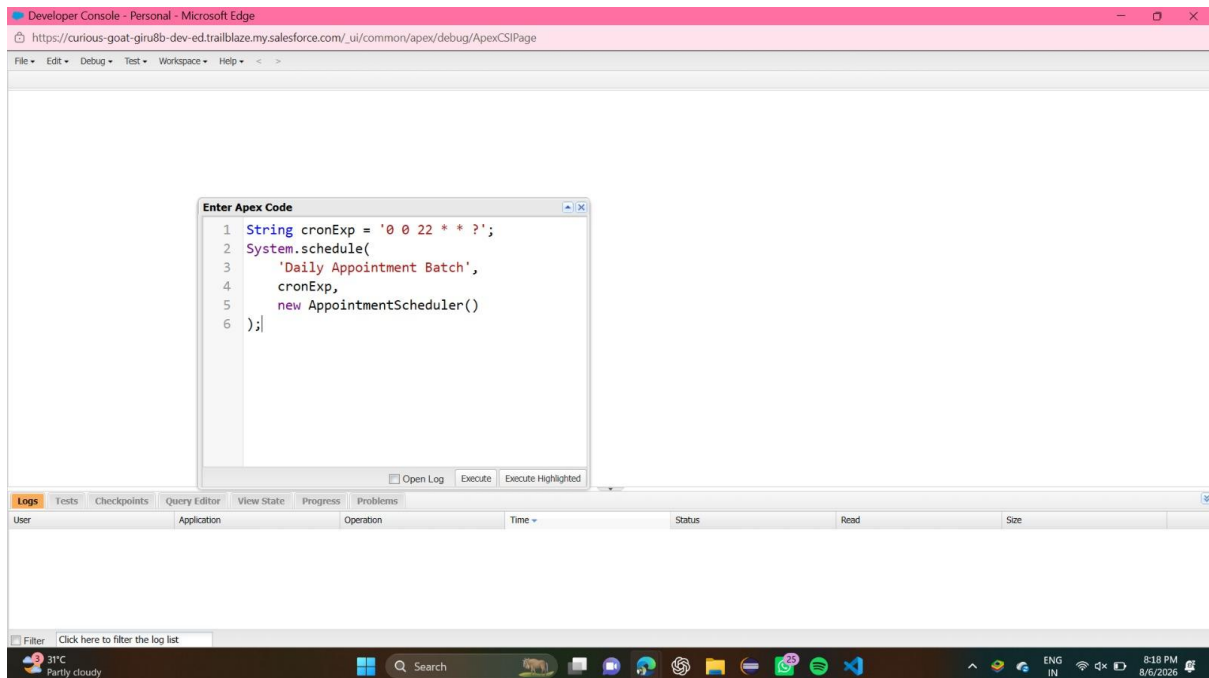
Objective

To schedule the Appointment Scheduler using a CRON expression for automatic execution.

Work Performed

An Apex scheduling script was executed through the Developer Console using the `System.schedule()` method. A CRON expression was defined to schedule the **AppointmentScheduler** class, allowing the appointment batch process to execute automatically at the specified time. This demonstrated how Salesforce schedules background jobs for recurring business operations.

Developer Console – Scheduling Apex using CRON Expression



Task – Verifying Scheduled Jobs

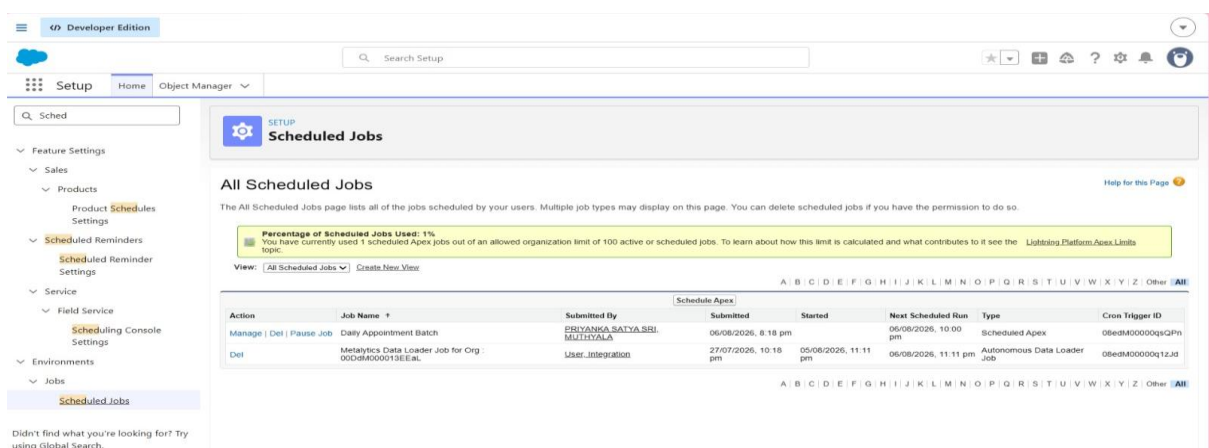
Objective

To verify that the scheduled Apex job was successfully created.

Work Performed

The Scheduled Jobs page in Salesforce Setup was accessed to verify the newly created scheduled job. The job named **Daily Appointment Batch** appeared successfully with its scheduled execution time, confirming that the scheduling process completed successfully.

Salesforce Setup – Scheduled Jobs



Task – Monitoring Scheduled Apex Execution

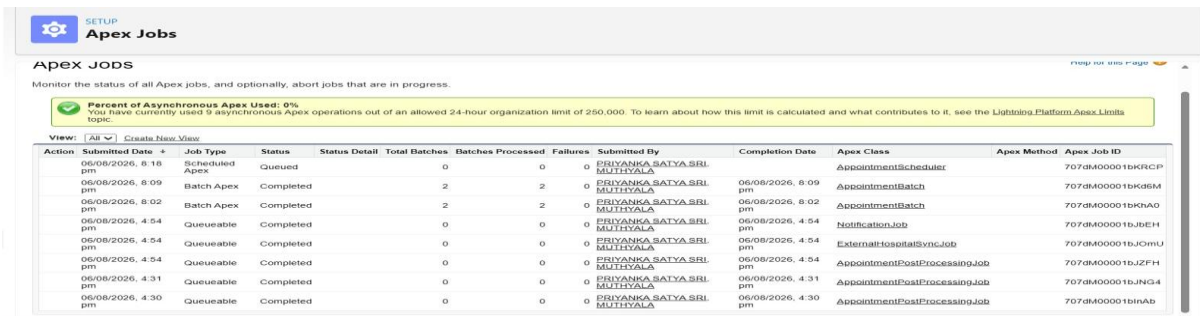
Objective

To monitor the execution status of asynchronous Apex jobs.

Work Performed

The Apex Jobs page was used to monitor the status of scheduled and asynchronous jobs. The execution history displayed the Scheduled Apex job along with previously executed Queueable Apex and Batch Apex jobs. This confirmed successful execution and provided a centralized view for monitoring asynchronous processing within Salesforce.

Salesforce Setup – Apex Jobs



The screenshot shows the Salesforce Apex Jobs page. At the top, there's a header with the Salesforce logo and 'Apex Jobs'. Below the header, there's a sub-header 'Apex Jobs' and a description: 'Monitor the status of all Apex jobs, and optionally, abort jobs that are in progress.' A green banner indicates 'Percent of Asynchronous Apex Used: 0%' and provides a link to 'Lightning Platform Apex Limits'. Below this, there's a table with columns: Action, Submitted Date, Job Type, Status, Status Detail, Total Batches, Batches Processed, Failures, Submitted By, Completion Date, Apex Class, Apex Method, and Apex Job ID. The table lists several jobs, including Scheduled Apex, Batch Apex, and Queueable Apex jobs, all of which are in a 'Completed' status.

Action	Submitted Date	Job Type	Status	Status Detail	Total Batches	Batches Processed	Failures	Submitted By	Completion Date	Apex Class	Apex Method	Apex Job ID
	05/08/2026, 8:18 pm	Scheduled Apex	Queued		0	0	0	PRIYANKA SATYA SRI MUTHIALA		AppointmentScheduler		707dM00001bKRCp
	05/08/2026, 8:09 pm	Batch Apex	Completed		2	2	0	PRIYANKA SATYA SRI MUTHIALA	05/08/2026, 8:09 pm	AppointmentBatch		707dM00001bKQ5M
	05/08/2026, 8:02 pm	Batch Apex	Completed		2	2	0	PRIYANKA SATYA SRI MUTHIALA	05/08/2026, 8:02 pm	AppointmentBatch		707dM00001bKHAQ
	05/08/2026, 4:54 pm	Queueable	Completed		0	0	0	PRIYANKA SATYA SRI MUTHIALA	05/08/2026, 4:54 pm	NotificationJob		707dM00001bJBEH
	05/08/2026, 4:54 pm	Queueable	Completed		0	0	0	PRIYANKA SATYA SRI MUTHIALA	05/08/2026, 4:54 pm	ExternalToSofiaSyncJob		707dM00001bJOMU
	05/08/2026, 4:54 pm	Queueable	Completed		0	0	0	PRIYANKA SATYA SRI MUTHIALA	05/08/2026, 4:54 pm	AppointmentPostProcessingJob		707dM00001bJZFh
	05/08/2026, 4:31 pm	Queueable	Completed		0	0	0	PRIYANKA SATYA SRI MUTHIALA	05/08/2026, 4:31 pm	AppointmentPostProcessingJob		707dM00001bJNG4
	05/08/2026, 4:30 pm	Queueable	Completed		0	0	0	PRIYANKA SATYA SRI MUTHIALA	05/08/2026, 4:30 pm	AppointmentPostProcessingJob		707dM00001bINAb

Conclusion

Engineering Sprint 8 provided practical experience in implementing Salesforce **Asynchronous Apex** using **Queueable Apex**, **Queueable Chaining**, **Batch Apex**, and **Scheduled Apex**. Through these engineering tasks, background processing, sequential job execution, large-scale record processing, and automated scheduling were successfully implemented and verified using Salesforce Developer Console, Apex Jobs, and Scheduled Jobs. These activities demonstrated how asynchronous processing improves application performance, enhances scalability, and follows Salesforce best practices for enterprise application development. Overall, this sprint strengthened practical knowledge of designing reliable, efficient, and maintainable asynchronous workflows capable of handling real-world business requirements in Salesforce.